

Orbital Mechanics with R

Simulating Planets, Binary Stars, Chaotic Systems, and the
N-Body Problem with `orbitr`

Dave Rosenman

Copyright © 2026 David P. Rosenman

All rights reserved. No part of this book may be reproduced in any form without written permission from the author, except for brief quotations in reviews.

The orbitr package is free software released under the MIT License; see <https://orbit-r.com>. All code listings in this book may be used freely.

Written for orbitr version 1.0.0.

First edition.

Table of contents

Preface

The documentation for `orbitr` lives at <https://orbit-r.com>, and it is free. It will show you how to put a planet around a star in four lines of R, how to load the whole solar system from real JPL data, and how to animate a planet orbiting two suns. If all you want is to drive the package, you may not need this book.

This book is about everything the documentation leaves out. The documentation says that `simulate_system()` uses the Velocity Verlet method by default because it “conserves energy over long timescales.” It does not say *why* Verlet conserves energy when the textbook Euler method does not, or where the method came from (a 1907 paper about the aurora, as it turns out), or how you would check for yourself that it is doing its job. The documentation gives you the circular-orbit speed $v_{\text{circ}} = \sqrt{GM/r}$ as a fact. It does not derive it, and it does not tell you how that one formula, pushed a little further, gives you Kepler’s three laws, the shape of every orbit in the solar system, and the velocity formulas that make the Kepler-16 example work.

So this is two books in one. The first is a physics book that uses `orbitr` the way a lab course uses a bench: every result gets derived on paper and then checked against a simulation you can run yourself. The second is an applications book that uses the physics to do real astrophysics: verifying Kepler’s third law across the solar system, building a binary star from conservation of momentum, watching Mars go retrograde from a geocentric frame, measuring how fast a chaotic triple system forgets its initial conditions.

Who this book is for

Two kinds of readers.

If you know R and are curious about how orbits work, this book will teach you the physics using tools you already have. You will need calculus and vectors, and it helps to have met Newton’s laws once, but you do not need to have taken a mechanics course. Where a derivation needs a fact from mechanics, the fact is stated and explained.

If you are a physics or astronomy student and want to *compute*, this book will show you how to turn the equations in your mechanics course into working simulations, and how to use tidy data tools to analyze what comes out. You do not need to know R at all. Appendix ?? gets you from nothing to running the book’s code (installing

Preface

R, packages, the pipe, the handful of dplyr and ggplot2 functions the book uses) in an hour, and points to the free books to read when you want more. Every piece of code in the chapters is explained, and the whole book uses a small, consistent vocabulary.

If the mathematics is the uncertain part, Appendix ?? collects everything the derivations assume (vectors and their products, derivatives of vectors, polar coordinates, Taylor series, the geometry of an ellipse) in one place, with the specific identities the book uses.

If you are both, you will find the two halves of the book talk to each other more than you might expect. The best way to understand a symplectic integrator is to watch a non-symplectic one fail; the best way to understand an orbital element is to vary it and look.

What is in this book

Part I, First Orbits, gets you running. Chapter 1 explains what an N-body simulation is and why anyone bothers when Kepler already solved the two-body problem. Chapter 2 walks through the four-line pipeline and the tidy table it produces.

Part II, Where the Physics Comes From, is the heart of the book. Chapter 3 starts from Newton's law and builds the N-body equations of motion that the package integrates. Chapter 4 solves the two-body problem by hand and derives everything the documentation states as fact: the orbit equation, Kepler's laws, the vis-viva equation, circular and escape velocity. Chapter 5 derives the three integrators (Euler, Euler-Cromer, Velocity Verlet), shows why only two of them are fit for orbits, and traces the Verlet method back through molecular dynamics, the aurora, and the *Principia*. Chapter 6 explains gravitational softening. Chapter 7 derives the conversion from Keplerian orbital elements to the Cartesian state vectors the engine actually uses.

Part III, Doing Astrophysics with orbitr, is the applications half. Each chapter takes a problem from astronomy, sets it up from the physics in Part II, runs it, and analyzes the result with dplyr and ggplot2.

Part IV, Under the Hood, looks at the C++ engine and at the places where a point-mass Newtonian model stops being enough.

The appendices collect the mathematical background the derivations assume, a quick function reference, the built-in physical constants with their sources, and solutions to the exercises.

What you need

- R, version 4.1 or later, from <https://cran.r-project.org>. The book uses R's native pipe `|>` everywhere.
- The `orbitr` package, version 1.0.0 or later (see the box below).
- The `dplyr` and `ggplot2` packages. They are installed automatically with `orbitr`. A few chapters also use `tidyr`.
- For the interactive 3D plots and the animations, the `plotly`, `gganimate`, and `gifski` packages. These are optional; the book tells you when it uses them.

An editor that runs R code chunks (RStudio or Positron, from <https://posit.co>) makes it easy to run the book's examples as you read.

Installing version 1.0.0

This book needs `orbitr` 1.0.0, which has been submitted to CRAN and will be available there any day now. Once it is, the usual command is all you need:

```
install.packages("orbitr")
```

If `packageVersion("orbitr")` still reports 0.3.0 after that, CRAN has not caught up yet. Install the current version directly from GitHub instead:

```
install.packages("remotes")
remotes::install_github("DRosenman/orbitr")
```

This builds the package's C++ engine on your machine, so on Windows you need Rtools (<https://cran.r-project.org/bin/windows/Rtools/>) and on a Mac the Xcode command-line tools (`xcode-select --install` in a terminal). Linux has a compiler already. Either route should leave `packageVersion("orbitr")` at 1.0.0 or higher.

The optional packages, in one line:

```
install.packages(c("tidyr", "plotly", "gganimate", "gifski"))
```

A note on versions

This book is written for `orbitr` 1.0.0, and every figure in it was produced by that version. Version 1.0 is the point at which the package's function names, arguments, and defaults became stable: anything that changes afterwards will be deprecated first, so code in this book should keep running on later versions, possibly

with a warning pointing at the replacement. Several of the analysis functions the book uses (`get_orbital_elements()`, `get_energy()`, `conserved_quantities()`, `continue_simulation()`, the barycentric reference frame) were added in 1.0.0 because this book needed them, so it will not run on 0.3.0. The physics in Part II does not depend on the version at all.

How the code works

Every figure and every number in this book was produced by the code shown next to it, when the book was built. Nothing was pasted in from elsewhere. That means two things for you as a reader. First, if you run the code, you should get the same picture. Second, when the text says a simulated period is “about 27 days,” that number came out of the simulation, not out of my head.

Code appears in blocks like this, with output marked by `#>`:

```
sqr(gravitational_constant * mass_earth / distance_earth_moon)
#> [1] 1018.289
```

Two kinds of output from `orbitr` do not survive the trip to paper. Interactive 3D plots (`plot_orbits_3d()`, and `plot_orbits()` whenever a body moves in z) are plotly widgets, and animations (`animate_system()`) are GIFs. Throughout the book I use static `ggplot2` figures, which are the same in every edition, and pass `three_d = FALSE` wherever a system has three-dimensional motion so that the figure stays flat.

Where the text discusses an animation or an interactive plot, the printed book shows the call you would make and a static snapshot from `plot_system()` in place of the result. The online edition at <https://book.orbit-r.com> shows them live.

Conventions

All quantities are in SI units: meters, kilograms, seconds. The built-in constants (`mass_earth`, `distance_earth_sun`, `speed_moon`, ...) are all SI, and all of the package’s arguments expect SI. Vectors are set in bold, $\mathbf{r}$, and their magnitudes in italic, $r = |\mathbf{r}|$. Unit vectors wear a hat, $\hat{\mathbf{r}} = \mathbf{r}/r$. Dots denote time derivatives: $\dot{\mathbf{r}}$ is velocity and $\ddot{\mathbf{r}}$ is acceleration.

The symbol μ always means the gravitational parameter $\mu = G(M + m)$ of a pair of bodies (or GM when one is much heavier). Some mechanics texts use μ for the reduced mass; this book writes the reduced mass as m_r to avoid the clash.

A few helper functions that the text defines for the derivations (measuring a period from a time series, solving Kepler’s equation, converting elements to a state vector by hand) are collected in the file `R/helpers.R` in the book’s source, which every chapter loads, so later chapters use them without repeating the definitions. Everything else is a package function.

Boxes marked **Where it comes from** give the history of a method or a formula: who found it, what problem they were working on, and how it ended up in the package. Boxes marked **Key result** collect the formulas you will reuse. Exercises close each chapter; solutions are in Appendix ??.

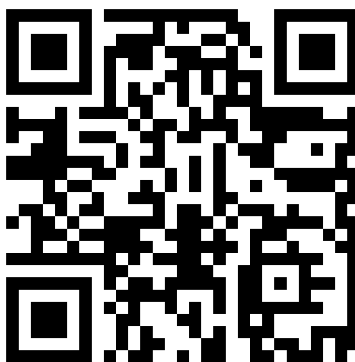
Acknowledgments

The physics in this book was shaped by Thornton and Marion’s *Classical Dynamics* (Thornton and Marion 2004) and Newman’s *Computational Physics* (Newman 2012), the two books I kept open while writing the engine. The N-body methods follow Aarseth (Aarseth 2003).

Try It First: The orbitr App

Before any code, it helps to have *seen* an orbit respond to a change: a planet given a little more speed and swinging out into an ellipse, a time step pushed too far and the orbit falling apart. The orbitr app is the package with a control panel on it. It runs in a browser, needs no installation, and does nothing the code in this book cannot do, which is the point: every control in it corresponds to an argument of a function you will meet in Chapter 2, and when you later type `add_body("Moon", mass = mass_moon, x = distance_earth_moon, vy = speed_moon)` you will already know what happens when that `vy` goes up. The app's **Code** tab makes that correspondence explicit: whatever system you have set up, it shows the R code that produces exactly what is on the screen. Set up a system by clicking, read the code, paste it into R, and you have run your first simulation.

The app is at <https://daverosenman.shinyapps.io/orbitr/>, or scan the code below. The online edition of this book at <https://book.orbit-r.com> embeds it in this page, with a guided walkthrough of what to try.



Ten minutes with it is a good investment before Chapter 2. Suggested experiments, in order: pick a preset system and run it as is; change one body's starting speed by ten percent in each direction and run again; shorten and lengthen the time step until something looks wrong; switch the integrator from Verlet to Euler and run the same system. After each one, open the Code tab and see which argument changed. Chapters 4 and 5 explain what you just saw.

Part I

First Orbits

1 Why Simulate Gravity?

In 1609 Johannes Kepler published the first two of his three laws of planetary motion, and in 1687 Isaac Newton showed that all three follow from a single inverse-square force law (Kepler 1609; Newton 1687). For one planet going around one star, the problem is *solved*: write down the initial position and velocity, and the orbit is an ellipse whose size, shape, and orientation you can compute by hand, exactly, for all time. Chapter 4 does exactly that.

So why would anyone simulate it?

1.1 Three reasons

Because two is a special number. Add a third body and the exact solution disappears. Newton knew it; he spent years on the Moon’s motion under the combined pull of the Earth and the Sun and reportedly said it was the only problem that ever made his head ache. In 1890 Henri Poincaré proved that the three-body problem has no general solution of the kind Kepler’s ellipses provide, and in doing so discovered what we now call chaos (Poincaré 1890). Our own solar system has one star, eight planets, hundreds of moons, and a great many smaller things, every one of which pulls on every other. The planets follow Kepler’s ellipses *almost*, and the “almost” contains all of the interesting physics: Jupiter tugging Earth’s orbit, resonances in the asteroid belt, the slow precession of Mercury’s perihelion. None of it can be written down in closed form. All of it can be simulated.

Because the solution and the understanding are different things. Even for the two-body problem, knowing that the answer is an ellipse is not the same as knowing what happens when you give a planet 20% too much speed, or launch it at the wrong angle, or put it around a star of twice the mass. A simulation lets you ask those questions in seconds and *see* the answer. Most of the intuition that working astronomers have about orbits was built this way, by changing something and looking.

Because the simulation is the check on the theory. When you derive, on paper, that a body launched at $\sqrt{2}$ times the circular speed should just barely escape, you can set up that exact situation and watch. If the simulation agrees, you understand both the physics and the code a little better. If it disagrees, one of them

1 Why Simulate Gravity?

is wrong, and finding out which is where the learning is. Part II of this book is built around that loop: derive, simulate, compare.

1.2 What a simulation actually is

Here is the whole idea in four sentences. At any instant, every body has a position and a velocity. Newton's law tells you the acceleration of every body from the positions of all the others. If you know where things are and how fast they are moving right now, you can estimate where they will be a short time Δt later. Do that, then do it again from the new state, and again, and you have an orbit.

The “estimate” is the whole subject of numerical integration, and it is where the craft is. The computer never solves the equations of motion. It takes small steps and hopes the error in each step is small enough, and cancels well enough, that the result after a million steps still means something. Whether that hope is justified depends entirely on *how* you take the step. Chapter 5 derives three ways of doing it, and shows that the most obvious one, the one every introductory text starts with, produces orbits that spiral outward and planets that quietly gain energy from nowhere. The one the package uses by default was invented for a different problem entirely and has a surprisingly deep reason for working.

This discreteness is the one thing to keep in mind every time you look at a simulation result. A simulated orbit is not the true orbit; it is the true orbit plus an error that depends on the step size and the method. Most of the time the error is invisible. Part of learning to simulate is learning when it is not.

1.3 What orbitr is

orbitr is an N-body gravitational simulator written for R (Rosenman 2026). You build a system by adding bodies to it with masses, positions, and velocities, run it forward in time, and get back a table. A few things make it different from the N-body codes that research astronomers use, and they are the reasons this book is possible:

- **The output is a tibble.** One row per body per time step, with columns for time, position, and velocity. There is no special result object to learn. Every tool in the R data-analysis ecosystem (dplyr, ggplot2, plotly, anything that takes a data frame) works on it directly. Most of the analysis in this book is a `filter()`, a `mutate()`, and a `ggplot()`.
- **It is built for the pipe.** A simulation reads top to bottom: create a system, add bodies, simulate, plot.

- **Analysis is built in.** The output can be re-centered on any body or on the system's center of mass, reduced to orbital elements, checked against the conservation laws, and continued from where it stopped, each with one function.
- **The physics is visible.** Three integration methods are available, so you can compare them. Softening is optional and off by default, so you get exact Newtonian gravity unless you ask otherwise. The gravitational constant is an argument, so you can turn gravity up, down, or off.
- **Real data is built in.** Masses, orbital distances, and speeds for the Sun, the planets, and the Moon are included as constants, and the planets' orbital elements come from the JPL DE440 ephemeris (Park et al. 2021), so `add_planet("Mars", parent = "Sun")` puts Mars where Mars actually is.
- **The inner loop is compiled.** The pairwise force calculation runs in C++ through Rcpp (Eddelbuettel and François 2011), with a pure-R fallback, so systems of a few dozen bodies simulate in seconds.

1.4 What orbitr is not

It is not an ephemeris. If you need to know where Mars will be on a given date to arc-second precision, use JPL Horizons, which fits decades of radar and spacecraft tracking data and includes effects this package ignores. `orbitr` starts from JPL's orbital elements at one epoch and integrates pure point-mass Newtonian gravity forward. Over a few years the planets land close to where they should; over centuries, small omitted effects accumulate.

The omissions are deliberate, and knowing them tells you what the package can and cannot answer:

- Bodies are point masses. Gravity outside a spherically symmetric body is exactly that of a point mass at its center (Chapter 3 proves this), so for orbital distances this costs nothing. But there is no notion of two bodies touching, no oblateness, no tides.
- Gravity is Newtonian. General relativity's correction to Mercury's perihelion precession, 43 arcseconds per century, is not in the model.
- There are no non-gravitational forces: no atmospheric drag, no radiation pressure, no thrust.
- The time step is fixed. Orbits that pass very close to a massive body need small steps near periapsis and would happily take large ones elsewhere; `orbitr` takes the same step throughout. Chapter 13 shows where this bites and how to handle it.

Chapter 16 returns to each of these, and to the package roadmap.

1.5 How to read this book

If you want to get something running first, go to Chapter 2 now, then come back. Part II can be read straight through as a physics text; the code is there to check the physics, not the other way round. Part III assumes Part II but each chapter stands alone, so you can go straight to binary stars or chaos if that is what you came for.

The exercises matter more than they might in a pure physics book, because the answers are something you can run. Many of them are of the form “predict, then simulate.” Do the prediction first.

1.6 Setting up

The preface lists what to install and how; the short version is R, `orbitr` 1.0.0 or later, and an editor that runs code chunks. Once that is done, compare your versions with the ones this book was built with:

```
packageVersion("orbitr")
#> [1] '1.0.0'
R.version.string
#> [1] "R version 4.6.1 (2026-06-24 ucrt)"
```

If `packageVersion("orbitr")` reports something older than 1.0.0, update: the analysis functions this book relies on were added in 1.0.0. Anything newer should run the book’s code unchanged; the changelog (`NEWS.md` in the GitHub repository, <https://github.com/DRosenman/orbitr>) lists any deprecations.

Exercises

Exercise 1.1. Kepler’s laws describe one body orbiting one other body. List three systems in the real solar system where you would expect them to be noticeably wrong, and say which other body is responsible in each case.

Exercise 1.2. The Moon orbits Earth in about 27.3 days. If a simulation steps forward one hour at a time, how many steps does one orbit take? How many if it steps one day at a time? Which do you think is more likely to give a good answer, and why? (Chapter 5 gives a rule of thumb; guess first.)

Exercise 1.3. Earth has a radius of about 6,371 km and the Moon orbits at about 384,400 km. The International Space Station orbits at about 420 km above the surface. For which of these two orbits do you think treating Earth as a point mass is a better approximation, and what feature of Earth might matter for the other?

2 Four Lines to an Orbit

This chapter gets you from nothing to a working simulation, then slows down and looks carefully at each piece. By the end you will know what every argument in the four-line pipeline does, what the table it produces looks like, and one subtle thing the four-line version gets slightly wrong, which turns out to be a preview of a real conservation law.

2.1 The Moon in four lines

```
library(orbitr)

create_system() |>
  add_body("Earth", mass = mass_earth) |>
  add_body("Moon", mass = mass_moon, x = distance_earth_moon, vy = speed_moon) |>
  simulate_system(time_step = seconds_per_hour, duration = seconds_per_day * 28) |>
  plot_orbits()
```

Earth sits at the origin. The Moon starts 384,400 km away along the x axis, moving at 1,022 m/s in the $+y$ direction, and 28 days later it has gone around once. (The Earth's own path is in there too; it is too small to see at this scale. We will come back to it.)

2.2 What each line does

create_system() makes an empty system. Its one argument is the gravitational constant, G , which defaults to the real value, $6.6743 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$. You will almost never change it, but it is there, and setting it to zero is a quick way to confirm that a body with no forces on it moves in a straight line.

add_body() adds one body. It needs a name (**id**) and a **mass**; everything else defaults to zero. The position arguments **x**, **y**, **z** are in meters and the velocity arguments **vx**, **vy**, **vz** in meters per second. Earth gets only a mass, so it starts at the origin at rest. The Moon gets a mass, an x position, and a y velocity. The constants

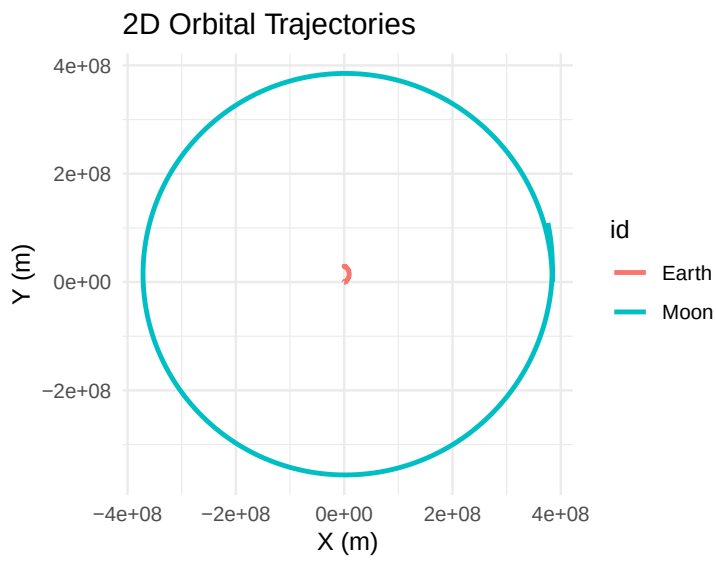


Figure 2.1: The Moon's orbit around Earth over 28 days, simulated one hour at a time.

`mass_earth`, `mass_moon`, `distance_earth_moon`, and `speed_moon` are exported by the package; Appendix ?? lists all of them with their sources.

`simulate_system()` runs the integration. `time_step` is the size of each step in seconds and `duration` is how long to run, also in seconds. Here that is one hour per step for 28 days, or $28 \times 24 = 672$ steps. The defaults are one hour and one year. Two more arguments matter and have sensible defaults: `method`, which picks the integrator ("verlet" by default; Chapter 5), and `softening`, which is zero by default, meaning exact Newtonian gravity (Chapter 6).

`plot_orbits()` draws the path of every body with `ggplot2` and returns the plot. Because it is an ordinary `ggplot` object you can add layers to it with `+`.

The constants `seconds_per_hour`, `seconds_per_day`, and `seconds_per_year` exist because every time argument in the package is in seconds, and nobody wants to type 2419200.

2.3 What came back

Drop the plotting step and keep the result:

```
moon <- create_system() |>
  add_body("Earth", mass = mass_earth) |>
  add_body("Moon", mass = mass_moon, x = distance_earth_moon, vy = speed_moon) |>
  simulate_system(time_step = seconds_per_hour, duration = seconds_per_day * 28)

moon
#> # A tibble: 1,346 x 9
#>   id      mass      x      y      z      vx      vy      vz      t
#>   <chr>   <dbl>   <dbl>   <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl>
#> 1 Earth  5.97e24      0      0      0      0      0      0      0
#> 2 Moon   7.34e22 384400000      0      0      0    1022      0      0
#> 3 Earth  5.97e24    215.      0      0    0.119    0.000571      0      0
#> 4 Moon   7.34e22 384382520. 3679200      0   -9.71    1022.      0      0
#> 5 Earth  5.97e24    860.     4.11      0    0.239    0.00229      0      0
#> 6 Moon   7.34e22 384330083. 7358065.      0  -19.4    1022.      0      0
#> 7 Earth  5.97e24   1934.    16.5      0    0.358    0.00514      0      0
#> 8 Moon   7.34e22 384242692. 11036262.      0  -29.1    1022.      0      0
#> 9 Earth  5.97e24   3438.    41.1      0    0.477    0.00914      0      0
#> 10 Moon  7.34e22 384120357. 14713454.      0  -38.8    1021.      0      0
#> # i 1,336 more rows
```

It is a tibble. Each row is one body at one instant. There are 1346 rows: two bodies, times 673 time steps, which is the 672 steps plus the initial state. The columns are

2 Four Lines to an Orbit

the body's `id`, its `mass`, its position `x`, `y`, `z`, its velocity `vx`, `vy`, `vz`, and `time` in seconds from the start.

That is the entire output format. There is nothing else to learn, which is the point: everything you already know how to do with a data frame works. How far is the Moon from the origin at the end of the run?

```
moon |>
  filter(time == max(time)) |>
  mutate(r_km = sqrt(x^2 + y^2 + z^2) / 1e3) |>
  select(id, time, r_km)
#> # A tibble: 2 x 3
#>   id      time    r_km
#>   <chr>   <dbl>   <dbl>
#> 1 Earth 2419200 29053.
#> 2 Moon 2419200 391550.
```

How does the Moon's distance change over the month?

```
library(dplyr)
library(ggplot2)

moon |>
  filter(id == "Moon") |>
  mutate(r_km = sqrt(x^2 + y^2 + z^2) / 1e3) |>
  ggplot(aes(time / seconds_per_day, r_km)) +
  geom_line() +
  labs(x = "Day", y = "Distance from origin (km)")
```

The distance dips and recovers: the orbit is an ellipse, not a circle, because 1,022 m/s is not exactly the speed that produces a circle at this distance. The speed that does is $\sqrt{G(M+m)/r}$, which evaluates to

```
sqrt(gravitational_constant * (mass_earth + mass_moon) / distance_earth_moon)
#> [1] 1024.529
```

m/s. Chapter 4 derives that formula and shows how the small gap between it and 1,022 m/s determines the eccentricity of the orbit in Figure ?? . For now, just notice that the simulation knows about it without being told.

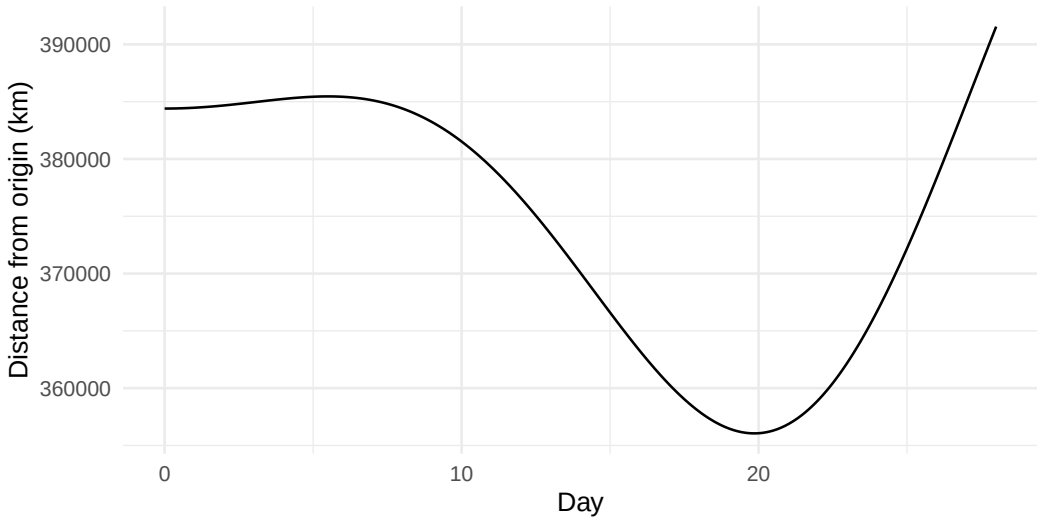


Figure 2.2: The Moon's distance from the origin over the 28-day run. It is not quite constant: the orbit is slightly elliptical.

2.4 Where did Earth go?

I said Earth's path was too small to see. Let's look at it on its own:

```
moon |>
  filter(id == "Earth") |>
  ggplot(aes(x / 1e3, y / 1e3)) +
  geom_path() +
  coord_equal() +
  labs(x = "x (km)", y = "y (km)")
```

Earth does not stay at the origin. It makes a small loop, as it should, because the Moon pulls on Earth just as Earth pulls on the Moon, and the two orbit their common center of mass. But the loop also *drifts*: after 28 days Earth has moved about

```
drift_km <- moon |>
  filter(id == "Earth", time == max(time)) |>
  mutate(r_km = sqrt(x^2 + y^2) / 1e3) |>
  pull(r_km)
round(drift_km)
#> [1] 29053
```

kilometers from where it started. Nothing is wrong with the simulation. The drift is the simulation being *right* about something we got wrong in the setup.

2 Four Lines to an Orbit

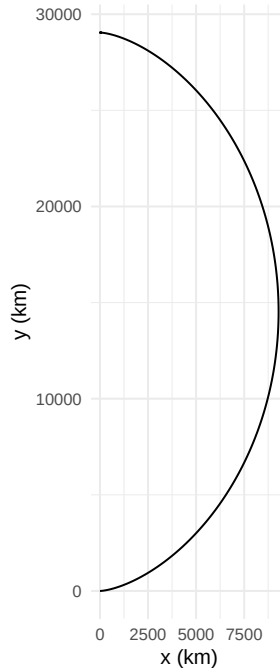


Figure 2.3: Earth’s own motion during the 28-day run. It loops once around the pair’s center of mass while the whole system drifts in the $+y$ direction.

We gave the Moon momentum, $m_{\text{Moon}} \times 1022$ m/s in the $+y$ direction, and gave Earth none. The total momentum of the system is therefore not zero, and since gravity is an internal force, total momentum is conserved: the system’s center of mass has to keep moving at a constant velocity forever. That velocity is the Moon’s momentum divided by the total mass,

```
v_com <- mass_moon * speed_moon / (mass_earth + mass_moon)
v_com
#> [1] 12.41192
```

about 12 m/s, which over 28 days is 3.0027×10^4 km. That is the drift in Figure ??, exactly.

The fix is to give Earth the equal and opposite momentum so the total is zero. Earth’s velocity must be $-v_{\text{Moon}} m_{\text{Moon}}/m_{\text{Earth}}$:

```
v_earth <- -speed_moon * mass_moon / mass_earth

moon_fixed <- create_system() |>
  add_body("Earth", mass = mass_earth, vy = v_earth) |>
  add_body("Moon", mass = mass_moon, x = distance_earth_moon, vy = speed_moon) |>
```

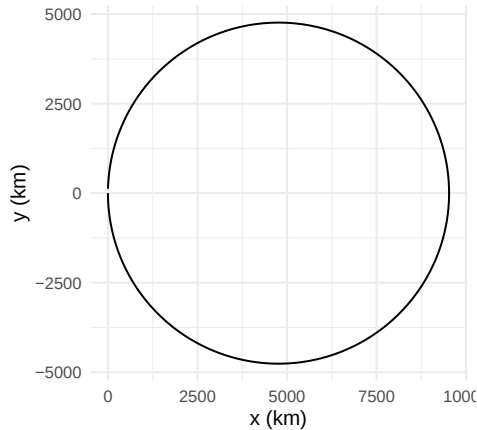


Figure 2.4: With zero total momentum, Earth’s path closes: a small circle around the center of mass, about 4,700 km from the origin.

```
simulate_system(time_step = seconds_per_hour, duration = seconds_per_day *
moon_fixed |>
  filter(id == "Earth") |>
  ggplot(aes(x / 1e3, y / 1e3)) +
  geom_path() +
  coord_equal() +
  labs(x = "x (km)", y = "y (km)")
```

Now Earth’s path closes. For a Sun-planet system the same drift exists and is a thousand times smaller, which is why the package’s `add_sun()` puts the Sun at rest at the origin and nobody notices. For a binary star, where the two masses are comparable, getting the momentum right is the whole problem, and Chapter 9 builds the Kepler-16 system from exactly this principle.

If you just want to *look* at the Moon going around Earth without thinking about the center of mass, there is a shortcut: re-center the data on Earth after the fact.

```
moon |>
  shift_reference_frame("Earth") |>
  plot_orbits()
```

`shift_reference_frame()` subtracts Earth’s position and velocity from every body at every time step, which puts Earth at the origin for the whole run. And if what you want is the drift gone without re-running anything, `shift_reference_frame(moon, "barycenter")` re-centers every step on the pair’s center of mass, which is where

the system was sitting all along. Chapter 10 is about what you can learn by doing this.

2.5 Real bodies without looking anything up

Typing masses, distances, and speeds works for any system you can imagine. For the real solar system there is an easier way. `add_sun()` places the Sun at the origin at rest, and `add_planet()` places a planet using its real orbital elements from the JPL DE440 ephemeris:

```
create_system() |>
  add_sun() |>
  add_planet("Mercury", parent = "Sun") |>
  add_planet("Venus",   parent = "Sun") |>
  add_planet("Earth",   parent = "Sun") |>
  add_planet("Mars",    parent = "Sun") |>
  simulate_system(time_step = seconds_per_day, duration = seconds_per_year) |>
  plot_orbits(three_d = FALSE)
```

`parent = "Sun"` tells `add_planet()` which body the orbit is around; the parent must already be in the system. Mercury completes about four orbits in the time Earth completes one, and you can see that its orbit is noticeably off-center: Mercury's eccentricity is 0.21, the largest of any planet.

Why `three_d = FALSE`?

The real planets' orbits are slightly tilted relative to one another, so once you use `add_planet()` the bodies have z motion. `plot_orbits()` notices and, by default, returns an interactive 3D plotly widget instead of a ggplot. That is the right thing on screen and the wrong thing on paper, so throughout this book I pass `three_d = FALSE` for any system with real orbital elements. On your own screen, leave it off and rotate the view.

`load_solar_system()` builds the Sun, all eight planets, the Moon, and Pluto in one call, and `remove_body()` drops the ones you do not want:

```
load_solar_system() |>
  remove_body(c("Moon", "Pluto")) |>
  simulate_system(time_step = seconds_per_day, duration = seconds_per_year * 12) |>
  plot_orbits(three_d = FALSE)
```

The Moon and Pluto are removed here for different reasons. Pluto's orbit is large and does not change the picture. The Moon is removed because simulating it correctly

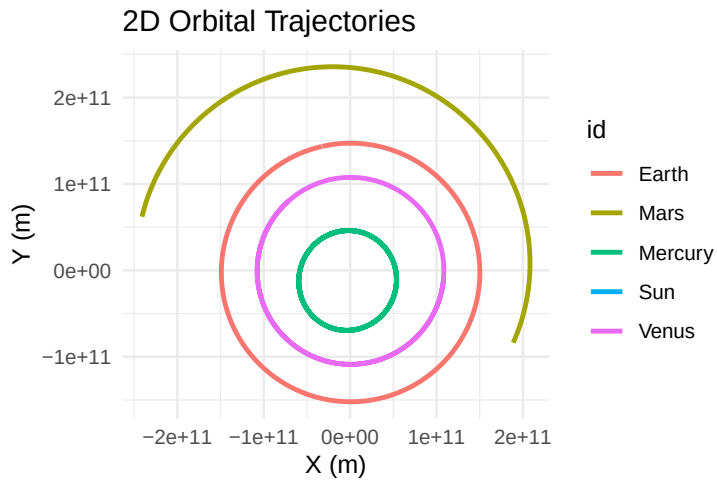


Figure 2.5: Mercury, Venus, Earth, and Mars for one Earth year, placed from JPL orbital elements.

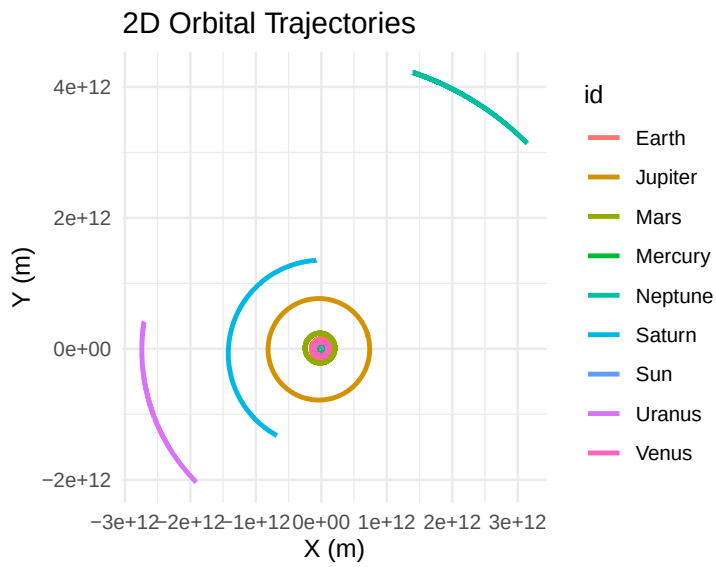


Figure 2.6: The whole solar system for twelve years. The inner planets are a smudge at this scale; Neptune has completed less than a tenth of an orbit.

needs a time step of an hour or so, and a twelve-year run at hourly steps is over a hundred thousand steps. One of the first things you learn about N-body simulation is that the time step is set by the *fastest* orbit in the system, not the one you care about. Chapter 5 explains why.

2.6 Looking before you leap

A system is an object you can inspect before you run it. Printing one shows the gravitational constant and the table of bodies:

```
sys <- create_system() |>
  add_sun() |>
  add_planet("Earth", parent = "Sun") |>
  add_planet("Mars", parent = "Sun")

sys
#>               orbit_system
#> G: 6.6743e-11 (standard)
#> Bodies: 3
#>
#> # A tibble: 3 x 8
#>   id      mass      x      y      z      vx      vy
#>   <chr>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl> <dbl>
#> 1 Sun    1.99e30      0      0      0      0      0
#> 2 Earth  5.97e24 -32965585121. 143360295956.      0 -29520. -6788.
#> 3 Mars   6.42e23 188789951360. -83706961610. -6395443036. 10750. 24226. 2
```

`get_bodies()` returns that table as a plain tibble, which is useful for checking that a hand-built system has the positions and velocities you intended:

```
get_bodies(sys) |>
  mutate(speed = sqrt(vx^2 + vy^2 + vz^2)) |>
  select(id, speed)
#> # A tibble: 3 x 2
#>   id      speed
#>   <chr>   <dbl>
#> 1 Sun      0
#> 2 Earth 30291.
#> 3 Mars  26505.
```

Earth's initial speed is a little over its mean orbital speed of 29,780 m/s, because `add_planet()` starts each body at periapsis (its closest approach to the Sun), where

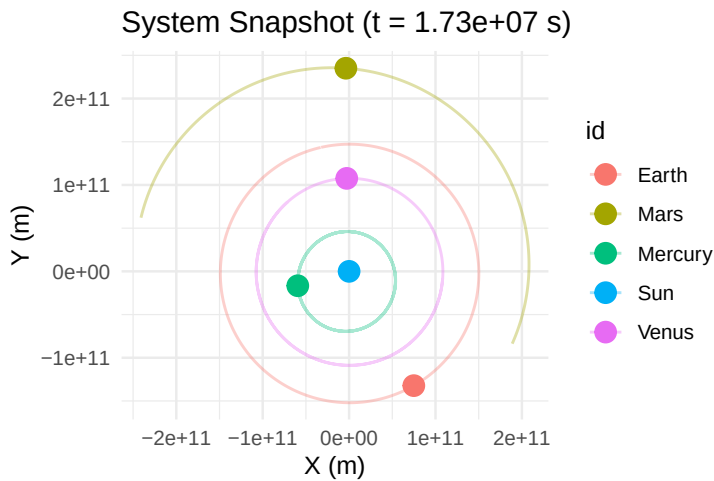


Figure 2.7: The inner solar system at day 200 of the run, with trails.

it moves fastest. The `nu` argument changes the starting point along the orbit; Chapter 7 covers it with the rest of the orbital elements.

2.7 Snapshots and animation

`plot_orbits()` draws whole trajectories. `plot_system()` draws where everything is at one instant, optionally with the trajectories drawn faintly behind:

```
inner <- create_system() |>
  add_sun() |>
  add_planet("Mercury", parent = "Sun") |>
  add_planet("Venus",    parent = "Sun") |>
  add_planet("Earth",    parent = "Sun") |>
  add_planet("Mars",     parent = "Sun") |>
  simulate_system(time_step = seconds_per_day, duration = seconds_per_year)

plot_system(inner, time = seconds_per_day * 200, trails = TRUE, three_d = FALSE)
```

The `time` argument is in simulation seconds and snaps to the nearest available step. A sequence of snapshots at different times is how the printed book shows motion. On screen, use the animation:

```
animate_system(inner, fps = 15, duration = 6, trails = TRUE)
```

This samples the run down to `fps * duration` frames and renders a GIF with `gganimate` (which you need to have installed, along with `gifski`). Each body leaves a fading wake. Rendering takes tens of seconds, so get the static plot right first.

2.8 Saving and sharing

`save_system()` writes a system, bodies and all, to an `.rds` file and `load_system()` reads it back, so a carefully tuned setup does not have to be rebuilt:

```
save_system(sys, "earth-mars.rds")
sys2 <- load_system("earth-mars.rds")
```

`export_bodies()` writes the body table to CSV, which is the right format for handing initial conditions to someone who is not using R:

```
export_bodies(sys, "earth-mars-bodies.csv")
```

The simulation output is just a tibble, so `readr::write_csv()` or `saveRDS()` handle it like any other data.

Exercises

Exercise 2.1. Build the Earth-Moon system with `create_system(G = 0)` and plot it. Explain the result in one sentence.

Exercise 2.2. Run the Moon simulation three times with the Moon's initial speed at 1,000, 1,022, and 1,044 m/s. Plot the Moon's distance from the origin against time for each run on one axis. Which run has the largest swing between nearest and farthest distance? Before you run it, predict whether the orbit will be larger or smaller than the circle in each case.

2 Four Lines to an Orbit

Exercise 2.3. In the four-line version, the center of mass drifts at $v_{\text{com}} = m_{\text{Moon}}v_{\text{Moon}}/(m_{\text{Earth}} + m_{\text{Moon}})$. Compute the center-of-mass position at every time step from the simulation tibble (mass-weighted mean of x and y , grouped by `time`) and plot its y coordinate against time. Confirm it is a straight line with the predicted slope.

Exercise 2.4. Re-run the inner solar system with `time_step = seconds_per_day * 5`. Does Mercury's orbit still look right? Try 10 days. At what step size does it visibly fall apart, and does Mars fall apart at the same step size?

Part II

Where the Physics Comes From

3 Newton's Law and the N-Body Problem

Everything orbitr does rests on one equation. This chapter writes it down carefully, in the vector form the code actually uses, and then asks the questions a careful reader should ask about it: why the mass of the body being pulled drops out, why a planet can be treated as a point, what the constant G is and how well it is known, and what the computer does with the equation once it has it. By the end of the chapter you will have written your own version of orbitr's inner loop in plain R and checked it against the package.

3.1 From the apple to the Moon

Newton's law of universal gravitation says that two point masses m_1 and m_2 separated by a distance r attract each other with a force of magnitude

$$F = \frac{G m_1 m_2}{r^2}. \quad (3.1)$$

The force is along the line joining them, and it falls off as the inverse square of the distance.

Where it comes from

The inverse-square law was in the air in the 1670s. Robert Hooke, Edmond Halley, and Christopher Wren all suspected that a force falling off as $1/r^2$ would produce Kepler's ellipses; none of them could prove it. In 1684 Halley asked Newton what orbit such a force would produce. Newton said an ellipse, that he had worked it out years before, and that he had lost the calculation. He redid it, and the redoing grew into the *Principia* (Newton 1687).

The famous test is the Moon. If the same force that makes an apple fall also holds the Moon in orbit, then since the Moon is about 60 Earth radii away, its acceleration toward Earth should be $1/60^2 = 1/3600$ of g . The acceleration needed to keep a body on a circle of radius r with period T is $4\pi^2 r/T^2$, and both numbers were known in Newton's time. Here is the comparison with modern values:

```

R_earth <- 6.371e6                                # m
g        <- 9.81                                   # m/s^2
T_moon   <- 27.32 * seconds_per_day                # sidereal month

g * (R_earth / distance_earth_moon)^2              # inverse-square prediction
#> [1] 0.002694744
4 * pi^2 * distance_earth_moon / T_moon^2          # centripetal, from the orbit
#> [1] 0.002723668

```

They agree to about one percent. The residual is mostly the Moon's own mass: the orbit's period is set by the combined mass, $G(M+m)$, which is 1.2% larger than GM alone. Chapter 4 makes this precise. Newton's own first calculation, in the 1660s, used an inaccurate figure for the size of the Earth and did not agree well; he returned to the problem years later when Picard's survey gave a better one.

3.2 The law in vector form

A simulation needs directions, not just magnitudes. Let $\mathbf{r}_j$ be the position vector of body j . The vector from body j to body k is $\mathbf{r}_k - \mathbf{r}_j$, its length is $r_{jk} = |\mathbf{r}_k - \mathbf{r}_j|$, and the unit vector pointing from j toward k is

$$\hat{\mathbf{r}}_{jk} = \frac{\mathbf{r}_k - \mathbf{r}_j}{r_{jk}}.$$

The force on j due to k points *toward* k (gravity attracts), so

$$\mathbf{F}_{jk} = \frac{G m_j m_k}{r_{jk}^2} \hat{\mathbf{r}}_{jk}. \quad (3.2)$$

Swapping j and k flips the unit vector and nothing else, so $\mathbf{F}_{kj} = -\mathbf{F}_{jk}$: Newton's third law is built in. This is the fact behind the drifting Earth in Chapter 2. Because every internal force comes in an equal and opposite pair, the total momentum of a system of gravitating bodies never changes, no matter how complicated the motion.

3.3 Superposition: the N-body acceleration

Newton's second law gives the acceleration of body j from the net force on it, $m_j \ddot{\mathbf{r}}_j = \sum_k \mathbf{F}_{jk}$. Gravitational forces add as vectors; the pull of the Sun on Earth does not

care that the Moon is also pulling. Summing Equation ?? over every other body and dividing by m_j ,

$$\ddot{\mathbf{r}}_j = \sum_{k \neq j} \frac{G m_k}{r_{jk}^2} \hat{\mathbf{r}}_{jk} = \sum_{k \neq j} G m_k \frac{\mathbf{r}_k - \mathbf{r}_j}{|\mathbf{r}_k - \mathbf{r}_j|^3}. \quad (3.3)$$

This is the equation. The first form is the one in the package documentation; the second is the one the code uses, because it avoids computing the unit vector separately. The cube in the denominator is one power for turning the difference vector into a unit vector and two for the inverse-square law.

Two things to notice.

The mass of body j has cancelled. The acceleration of a body in a gravitational field does not depend on its own mass. A feather and a hammer fall together; the International Space Station and an astronaut floating inside it follow the same orbit. This is only true because the m_j in Newton’s second law (inertial mass, the resistance to being accelerated) is the same quantity as the m_j in the law of gravitation (gravitational mass, the thing that gravity pulls on). Nothing in Newtonian mechanics requires those to be equal; they just are. The equality has been tested to about one part in 10^{15} (Touboul et al. 2022), and Einstein took it as the starting point of general relativity. For the simulation it means something practical: a body’s mass matters for how it pulls on the others, but a test particle of negligible mass follows exactly the same path it would with any other negligible mass.

The sum is over every pair. With N bodies there are $N(N - 1)$ ordered pairs, and each needs a subtraction, a norm, and a division. Doubling the number of bodies quadruples the work. This $O(N^2)$ scaling is the single fact that dominates the design of every N-body code ever written, and it is why orbitr’s inner loop is in C++ (Chapter 15). For the handful of bodies in this book it is not a problem.

3.4 Why point masses are enough

Earth is not a point. It is a ball 12,742 km across, and every piece of it pulls on the Moon from a slightly different direction and distance. Why is it legitimate to write Equation ?? with a single $\mathbf{r}_{\text{Earth}}$?

Theorem 3.1 (Newton’s shell theorem). *A spherically symmetric body attracts an external point mass exactly as if all of its mass were concentrated at its center. A spherically symmetric shell exerts no net gravitational force on a point mass anywhere inside it.*

3 Newton's Law and the N-Body Problem

Newton proved this geometrically in the *Principia* (Book I, Propositions 70 and 71), and he is often said to have delayed publishing the theory of gravitation until he had the proof, since without it the Moon test above is not justified. The modern proof takes three lines if you know Gauss's law for gravity, which says that the flux of the gravitational field $\mathbf{g}$ through any closed surface is proportional to the mass enclosed:

$$\oint \mathbf{g} \cdot d\mathbf{A} = -4\pi G M_{\text{enc}}.$$

Take the surface to be a sphere of radius r centered on the body. By symmetry, $\mathbf{g}$ points radially and has the same magnitude $g(r)$ everywhere on the sphere, so the integral is $-g(r) \cdot 4\pi r^2$ (negative because the field points inward). Hence $g(r) = GM_{\text{enc}}/r^2$, directed toward the center. Outside the body, M_{enc} is the whole mass, and the field is that of a point mass. Inside a shell, $M_{\text{enc}} = 0$, and the field vanishes.

So for any body whose mass is arranged in spherical shells, which is a very good description of stars and planets, the point-mass equation is not an approximation at all. It is exact, as long as the bodies do not overlap.

Where it fails is instructive. Earth bulges at the equator by about 21 km because it spins; the extra mass there pulls satellites in low orbit slightly off the point-mass prediction, and the orbits of GPS satellites precess measurably because of it. Astronomers call the leading correction J_2 . Two stars in a close binary raise tides on each other and are not spherical either. `orbitr` ignores all of this, which is the right choice for orbital distances and the wrong one for anything skimming a surface. The package roadmap lists J_2 as a possible addition; Chapter 16 discusses what it would take.

3.5 Units, G , and how well we know it

Everything in `orbitr` is in SI: meters, kilograms, seconds. The gravitational constant is

```
gravitational_constant  
#> [1] 6.6743e-11
```

in $\text{m}^3 \text{kg}^{-1} \text{s}^{-2}$. That is the CODATA 2018 recommended value, $6.67430(15) \times 10^{-11}$ (Tiesinga et al. 2021), and the digits in parentheses are the uncertainty in the last two places: about 2 parts in 10^5 . By the standards of fundamental constants that is terrible. The speed of light is exact by definition; Planck's constant is now exact by definition; the fine-structure constant is known to a few parts in 10^{10} . G is hard

to measure because gravity is so weak that laboratory experiments have to detect the pull of a kilogram-scale mass on another, and because there is no way to shield the apparatus from everything else.

The constant that astronomy actually uses is not G but the product GM for each body, called the gravitational parameter and written μ . For the Sun, $GM_\odot$ is known to about one part in 10^{10} , because it is what you get directly from fitting planetary orbits and spacecraft trajectories, no laboratory required. When the package reports the Sun's mass as 1.989×10^{30} kg, that number is $GM_\odot$ divided by a poorly known G , and its uncertainty is essentially the uncertainty in G . None of this affects a simulation, because the equations of motion only ever contain G and a mass together. It does mean that if you ever compare a simulated orbital period to a measured one at the fifth significant figure, the discrepancy may be in G , not in the integrator.

3.6 The equations of motion as a system

Equation ?? is a second-order differential equation for each of N bodies. A simulation does not work with second-order equations; it works with a *state* and a rule for advancing it. The state of the system at time t is the list of all positions and velocities,

$$(\mathbf{r}_1, \dots, \mathbf{r}_N, \mathbf{v}_1, \dots, \mathbf{v}_N),$$

$6N$ numbers in all. The rule for how the state changes is a pair of first-order equations for each body,

$$\dot{\mathbf{r}}_j = \mathbf{v}_j, \quad \dot{\mathbf{v}}_j = \mathbf{a}_j(\mathbf{r}_1, \dots, \mathbf{r}_N), \quad (3.4)$$

where $\mathbf{a}_j$ is the right-hand side of Equation ?. The accelerations depend only on positions, not on velocities. That is a special feature of gravity (and of any force that comes from a potential), and Chapter 5 will show that the best integrators exploit it.

Here is the structure of every N-body code, including this one, in pseudo-code:

```
state <- initial positions and velocities
for each time step:
  a      <- accelerations(state)           # the N-body sum, all pairs
  state <- advance(state, a, dt)           # the integrator
  record state
```

The `accelerations()` function is the physics. The `advance()` function is the numerics. `orbitr` keeps them separate, and so will this book: this chapter is about the first function, Chapter 5 is about the second.

3.7 Writing the acceleration yourself

The best way to be sure what Equation ?? says is to implement it. Here it is in plain R, with no attempt at speed:

```
accelerations <- function(bodies, G = gravitational_constant) {
  n <- nrow(bodies)
  pos <- as.matrix(bodies[, c("x", "y", "z")])
  a <- matrix(0, nrow = n, ncol = 3)
  for (j in seq_len(n)) {
    for (k in seq_len(n)) {
      if (j == k) next
      d <- pos[k, ] - pos[j, ]           # vector from j toward k
      r <- sqrt(sum(d^2))                 # distance
      a[j, ] <- a[j, ] + G * bodies$mass[k] * d / r^3
    }
  }
  tibble(id = bodies$id, ax = a[, 1], ay = a[, 2], az = a[, 3])
}
```

The double loop runs over all ordered pairs (j, k) with $j \neq k$. For each pair it forms the difference vector, its length, and the term in the sum. `bodies` is any data frame with columns `id`, `mass`, `x`, `y`, `z`, which is exactly what `get_bodies()` returns:

```
em <- create_system() |>
  add_body("Earth", mass = mass_earth) |>
  add_body("Moon", mass = mass_moon, x = distance_earth_moon, vy = speed_moon)

accelerations(get_bodies(em))
#> # A tibble: 2 x 4
#>   id      ax      ay      az
#>   <chr>   <dbl> <dbl> <dbl>
#> 1 Earth  0.0000332    0      0
#> 2 Moon  -0.00270      0      0
```

The Moon's acceleration is in the $-x$ direction, toward Earth, with magnitude GM_{Earth}/r^2 :

```
gravitational_constant * mass_earth / distance_earth_moon^2
#> [1] 0.002697483
```

and Earth's is in the $+x$ direction, toward the Moon, smaller by the mass ratio $m_{\text{Moon}}/M_{\text{Earth}} \approx 1/81$. Equal and opposite *forces*; unequal accelerations.

Now a check against the package. The package's output does not include accelerations, but it does include velocities at every step, and over one hour the velocity change of the Moon should be close to $\mathbf{a}\Delta t$:

```
sim <- simulate_system(em, time_step = seconds_per_hour,
                      duration = seconds_per_hour)

sim |>
  filter(id == "Moon") |>
  summarise(dvx_per_second = diff(vx) / seconds_per_hour)
#> # A tibble: 1 x 1
#>   dvx_per_second
#>   <dbl>
#> 1      -0.00270
```

That is the x acceleration computed above, to about four significant figures. (It is not exact because the acceleration changes slightly during the hour as the Moon moves; the integrator uses the average of the accelerations at the start and end of the step, which is one of the things that makes it good.)

3.8 A three-body example

With three bodies the sums in Equation ?? have two terms each. Put the Sun at the origin, Earth at 1 AU, and the Moon 384,400 km beyond Earth, and ask how hard the Sun pulls on the Moon compared to how hard Earth does:

```
sem <- create_system() |>
  add_sun() |>
  add_body("Earth", mass = mass_earth, x = distance_earth_sun) |>
  add_body("Moon", mass = mass_moon,
          x = distance_earth_sun + distance_earth_moon)

a_sun_on_moon <- gravitational_constant * mass_sun /
  (distance_earth_sun + distance_earth_moon)^2
a_earth_on_moon <- gravitational_constant * mass_earth /
  distance_earth_moon^2
```

```
a_sun_on_moon / a_earth_on_moon  
#> [1] 2.187709
```

The Sun pulls on the Moon more than twice as hard as Earth does. This surprises almost everyone the first time. The Moon orbits Earth not because Earth's pull is stronger but because Earth and the Moon are *both* falling toward the Sun together, at nearly the same rate; what keeps the Moon bound to Earth is the small *difference* between the Sun's pull on the two of them. Seen from the Sun, the Moon's path is a slightly wobbly ellipse that is everywhere concave toward the Sun. Chapter 10 draws it.

The full acceleration of the Moon is the vector sum of both terms, and our function gets it right with no changes:

```
accelerations(get_bodies(sem)) |>  
  filter(id == "Moon")  
#> # A tibble: 1 x 4  
#>   id          ax      ay      az  
#>   <chr>      <dbl> <dbl> <dbl>  
#> 1 Moon    -0.00860      0      0
```

3.9 What the engine does differently

The function above is correct and slow. Each call does $N(N - 1)$ iterations of interpreted R, and a simulation calls it once or twice per step for hundreds of thousands of steps. The package's pure-R fallback vectorizes the same computation with matrix outer products so that R does the loop in compiled code internally; its C++ engine writes the double loop directly in C++ and runs about as fast as the hardware allows. Both compute exactly Equation ??, with one small addition: an optional softening length ε that replaces r^3 in the denominator with $(r^2 + \varepsilon^2)^{3/2}$. With $\varepsilon = 0$, the default, there is no difference. Chapter 6 explains what softening is for.

Key results

- The acceleration of body j is $\ddot{\mathbf{r}}_j = \sum_{k \neq j} G m_k \frac{\mathbf{r}_k - \mathbf{r}_j}{|\mathbf{r}_k - \mathbf{r}_j|^3}$. It does not depend on m_j .
- Internal forces come in equal and opposite pairs, so total momentum is conserved: the center of mass of any isolated system moves at constant velocity.

- A spherically symmetric body gravitates exactly like a point mass at its center (shell theorem).
- The computational cost grows as N^2 .

Exercises

Exercise 3.1. Using the constants in the package, compute the acceleration of Earth due to the Sun, due to the Moon, and due to Jupiter at its closest approach (Jupiter's distance from the Sun minus Earth's). Express the last two as fractions of the first.

Exercise 3.2. A hollow spherical shell of mass M and radius R has a small hole drilled in it. A pebble is dropped into the hole. Describe its motion inside the shell. (No calculation needed, but say which theorem you are using.)

Exercise 3.3. Rewrite `accelerations()` without the inner loop: for a fixed j , compute the difference vectors to all other bodies at once with `sweep()` or by subtracting a row from the matrix, and sum the contributions with `colSums()`. Check that it gives the same answer on the Sun-Earth-Moon system.

Exercise 3.4. Simulate the Earth-Moon system for 28 days with `create_system(G = gravitational_constant * 1.01)` and again with the standard value. By how much does the Moon's position differ at the end? Is a 1% change in G detectable from one month of orbit data?

4 The Two-Body Problem, Solved by Hand

The package documentation hands you a handful of formulas. The circular-orbit speed is $\sqrt{GM/r}$; escape speed is $\sqrt{2}$ times that; an orbit's period is $2\pi\sqrt{a^3/\mu}$; the distance from the parent at true anomaly ν is $a(1 - e^2)/(1 + e \cos \nu)$. Every one of them is true, and every one of them follows from Newton's law in a few lines of vector algebra. This chapter does the algebra. It is the one place in the book where the derivations come first and the code comes second, and it will repay the effort: these are the formulas you will use to set up every system in Part III.

The strategy is the physicist's standard one. First find the quantities that *do not change* as the bodies move, the conserved quantities. Then use them to pin down the shape of the orbit without ever integrating anything. Along the way, Kepler's three laws drop out as consequences.

4.1 Two bodies become one

Take two point masses, m_1 at $\mathbf{r}_1$ and m_2 at $\mathbf{r}_2$, with nothing else in the universe. From Equation ??,

$$\ddot{\mathbf{r}}_1 = Gm_2 \frac{\mathbf{r}_2 - \mathbf{r}_1}{r^3}, \quad \ddot{\mathbf{r}}_2 = Gm_1 \frac{\mathbf{r}_1 - \mathbf{r}_2}{r^3},$$

where $r = |\mathbf{r}_2 - \mathbf{r}_1|$. Subtract the first from the second. Writing $\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1$ for the *relative* position of body 2 as seen from body 1,

$$\ddot{\mathbf{r}} = -\frac{\mu}{r^3} \mathbf{r}, \quad \mu = G(m_1 + m_2). \quad (4.1)$$

Two coupled equations in six coordinates have become one equation in three. It says the relative position vector accelerates toward the origin as if there were a single fixed mass $m_1 + m_2$ sitting there. The combination $\mu = G(m_1 + m_2)$ is called the gravitational parameter, and it is the only combination of G and the masses that the relative motion knows about.

What happened to the other three coordinates? Add m_1 times the first equation to m_2 times the second: the right-hand sides cancel, so the center of mass $\mathbf{R} =$

4 The Two-Body Problem, Solved by Hand

$(m_1\mathbf{r}_1 + m_2\mathbf{r}_2)/(m_1 + m_2)$ has $\ddot{\mathbf{R}} = 0$. It moves in a straight line at constant velocity, which is the drift we met in Chapter 2. Given $\mathbf{R}$ and $\mathbf{r}$, the individual positions are

$$\mathbf{r}_1 = \mathbf{R} - \frac{m_2}{m_1 + m_2} \mathbf{r}, \quad \mathbf{r}_2 = \mathbf{R} + \frac{m_1}{m_1 + m_2} \mathbf{r}. \quad (4.2)$$

Both bodies trace the same shape of orbit around the center of mass, scaled by the mass ratio. In Chapter 2's fixed Earth-Moon system, Earth's little circle had a radius $m_{\text{Moon}}/(m_{\text{Earth}} + m_{\text{Moon}}) \approx 1/82$ of the Moon's.

When one body is much heavier, $m_1 \gg m_2$, two things happen at once: $\mu \approx Gm_1$, and $\mathbf{r}_1 \approx \mathbf{R}$, so the heavy body sits essentially at the center of mass and the light body orbits it. That is the “fixed central body” picture of the two-body vignette, and the approximation is excellent for the Sun and any planet (the Sun outweighs Jupiter a thousand to one) and merely good for Earth and the Moon, where the correction is 1.2%. From here on the derivation is exact for the relative motion; whether you then set $\mu = GM$ or $\mu = G(M + m)$ is a choice about how much precision you need.

4.2 Angular momentum is conserved, and the orbit is flat

Define the *specific angular momentum* (angular momentum per unit mass) of the relative motion,

$$\mathbf{h} = \mathbf{r} \times \dot{\mathbf{r}}.$$

Differentiate it using the product rule:

$$\dot{\mathbf{h}} = \dot{\mathbf{r}} \times \dot{\mathbf{r}} + \mathbf{r} \times \ddot{\mathbf{r}} = \mathbf{0} + \mathbf{r} \times \left(-\frac{\mu}{r^3} \mathbf{r}\right) = \mathbf{0}.$$

The first term is a vector crossed with itself and the second is $\mathbf{r}$ crossed with a multiple of $\mathbf{r}$; both vanish. So $\mathbf{h}$ is a constant of the motion. This works for *any* force directed along $\mathbf{r}$, not just inverse-square: it is a consequence of the force being central.

Two consequences follow immediately. Since $\mathbf{h}$ is perpendicular to both $\mathbf{r}$ and $\dot{\mathbf{r}}$ at every instant, and $\mathbf{h}$ never changes, the position and velocity are confined forever to the plane perpendicular to $\mathbf{h}$. **Every two-body orbit is planar.** And the magnitude $h = |\mathbf{r} \times \dot{\mathbf{r}}|$ is constant. In a small time dt the position vector sweeps out

a thin triangle with sides $\mathbf{r}$ and $\dot{\mathbf{r}} dt$, whose area is half the magnitude of their cross product:

$$\frac{dA}{dt} = \frac{1}{2} |\mathbf{r} \times \dot{\mathbf{r}}| = \frac{h}{2}. \quad (4.3)$$

The line from the central body to the orbiting one sweeps out equal areas in equal times. That is **Kepler's second law**, and it took one line.

4.3 Energy is conserved

Now dot Equation ?? with the velocity:

$$\dot{\mathbf{r}} \cdot \ddot{\mathbf{r}} = -\frac{\mu}{r^3} \mathbf{r} \cdot \dot{\mathbf{r}}.$$

The left side is $\frac{d}{dt} (\frac{1}{2} \dot{\mathbf{r}} \cdot \dot{\mathbf{r}}) = \frac{d}{dt} (\frac{1}{2} v^2)$. For the right side, note that $r^2 = \mathbf{r} \cdot \mathbf{r}$, so $2r\dot{r} = 2\mathbf{r} \cdot \dot{\mathbf{r}}$ and $\mathbf{r} \cdot \dot{\mathbf{r}} = r\dot{r}$. Then $-\mu \mathbf{r} \cdot \dot{\mathbf{r}} / r^3 = -\mu \dot{r} / r^2 = \frac{d}{dt} (\mu / r)$. Moving everything to one side,

$$\frac{d}{dt} \left(\frac{v^2}{2} - \frac{\mu}{r} \right) = 0, \quad \varepsilon \equiv \frac{v^2}{2} - \frac{\mu}{r} = \text{const.} \quad (4.4)$$

ε is the *specific orbital energy*: kinetic energy per unit mass plus gravitational potential energy per unit mass. If $\varepsilon < 0$ the body cannot reach $r = \infty$ (it would need negative kinetic energy there), so it is bound. If $\varepsilon \geq 0$ it can, and it escapes.

Chapter 12 turns both of these conservation laws into diagnostics: a simulation that lets ε or $\mathbf{h}$ drift is a simulation with a problem.

4.4 The orbit equation

Here is the step that makes the whole chapter work. Conservation of $\mathbf{h}$ and ε constrains the motion but does not by itself give the shape of the path. For the inverse-square force, and essentially only for it, there is a *third* conserved vector, and it hands you the orbit directly.

Start from the time derivative of $\dot{\mathbf{r}} \times \mathbf{h}$. Since $\mathbf{h}$ is constant,

$$\frac{d}{dt} (\dot{\mathbf{r}} \times \mathbf{h}) = \ddot{\mathbf{r}} \times \mathbf{h} = -\frac{\mu}{r^3} \mathbf{r} \times (\mathbf{r} \times \dot{\mathbf{r}}).$$

Expand the triple product with the identity $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})$:

$$\mathbf{r} \times (\mathbf{r} \times \dot{\mathbf{r}}) = \mathbf{r} (\mathbf{r} \cdot \dot{\mathbf{r}}) - \dot{\mathbf{r}} r^2 = \mathbf{r} r \dot{r} - \dot{\mathbf{r}} r^2,$$

so

$$\frac{d}{dt} (\dot{\mathbf{r}} \times \mathbf{h}) = \mu \left(\frac{\dot{\mathbf{r}}}{r} - \frac{\dot{r} \mathbf{r}}{r^2} \right) = \mu \frac{d}{dt} \left(\frac{\mathbf{r}}{r} \right).$$

The last equality is just the quotient rule applied to $\mathbf{r}/r$. Both sides are total time derivatives, so their difference is a constant vector. Divide by μ and call the constant $\mathbf{e}$:

$$\mathbf{e} = \frac{\dot{\mathbf{r}} \times \mathbf{h}}{\mu} - \frac{\mathbf{r}}{r}. \quad (4.5)$$

This is the *eccentricity vector* (also called the Laplace-Runge-Lenz vector, in various normalizations). It lies in the orbital plane, because both terms do. To see what it means, take its dot product with $\mathbf{r}$. The first term gives $\mathbf{r} \cdot (\dot{\mathbf{r}} \times \mathbf{h})/\mu = \mathbf{h} \cdot (\mathbf{r} \times \dot{\mathbf{r}})/\mu = h^2/\mu$, using the cyclic property of the scalar triple product; the second gives $-r$. If ν is the angle between $\mathbf{e}$ and $\mathbf{r}$, the dot product is also $e r \cos \nu$. So

$$e r \cos \nu = \frac{h^2}{\mu} - r \quad \implies \quad r = \frac{h^2/\mu}{1 + e \cos \nu}. \quad (4.6)$$

This is the **orbit equation**: the distance from the central body as a function of the angle ν measured from the direction of $\mathbf{e}$. The quantity $p = h^2/\mu$ is called the semi-latus rectum, and $e = |\mathbf{e}|$ is the eccentricity. The distance r is smallest when $\cos \nu = 1$, so $\mathbf{e}$ points toward the point of closest approach, the *periapsis*, and ν , the *true anomaly*, is the angle from periapsis. Equation ?? is a conic section with one focus at the origin:

Table 4.1: The four kinds of two-body orbit.

	e	ε	shape	$r_{\max}$
0		$-\mu^2/2h^2$	circle	$= r_{\min}$
$0 < e < 1$		< 0	ellipse	finite
1		0	parabola	∞
> 1		> 0	hyperbola	∞

Bound orbits are ellipses with the central body at a focus. That is **Kepler's first law**, and it has now been derived from the inverse-square law, which is the thing Halley asked Newton about in 1684.

The relation between e and ε in the table comes from squaring Equation ?? . Since $\dot{\mathbf{r}} \perp \mathbf{h}$, $|\dot{\mathbf{r}} \times \mathbf{h}| = v h$, and the cross term is $-2\mathbf{r} \cdot (\dot{\mathbf{r}} \times \mathbf{h})/(\mu r) = -2h^2/(\mu r)$, so

$$e^2 = \frac{v^2 h^2}{\mu^2} - \frac{2h^2}{\mu r} + 1 = 1 + \frac{2\varepsilon h^2}{\mu^2}. \quad (4.7)$$

Negative energy means $e < 1$: bound orbits are ellipses, as the table says.

Where it comes from

Kepler extracted his first two laws from Tycho Brahe's observations of Mars, published in *Astronomia Nova* in 1609 (Kepler 1609), and the third from the periods and distances of all the planets, published in *Harmonices Mundi* in 1619 (Kepler 1619). He had no theory of why; he had tried circles and epicycles for years and the data would not fit. Newton's *Principia* (Newton 1687) showed that all three laws follow from an inverse-square force (Book I, Propositions 11 to 15), using geometry rather than vectors. The vector form here is the standard modern one; Goldstein (Goldstein et al. 2002) and Bate, Mueller, and White (Bate et al. 1971) both present it, and the eccentricity vector is the starting point of every algorithm that turns a position and velocity into orbital elements, including the one in Chapter 7.

4.5 The geometry of an ellipse

An ellipse has a longest diameter, the major axis, of length $2a$; a is the semi-major axis. From Equation ?? the nearest and farthest points are at $\nu = 0$ and $\nu = \pi$:

$$r_p = \frac{p}{1+e}, \quad r_a = \frac{p}{1-e}, \quad a = \frac{r_p + r_a}{2} = \frac{p}{1-e^2}.$$

So $p = a(1 - e^2)$, and the orbit equation takes the form the package uses:

$$r = \frac{a(1 - e^2)}{1 + e \cos \nu}, \quad r_p = a(1 - e), \quad r_a = a(1 + e). \quad (4.8)$$

The semi-minor axis is $b = a\sqrt{1 - e^2}$. Two more facts fall out of these. First, given any two-body orbit, you can read off its eccentricity from its extreme distances alone:

$$e = \frac{r_a - r_p}{r_a + r_p}. \quad (4.9)$$

4 The Two-Body Problem, Solved by Hand

Second, substituting $h^2 = \mu p = \mu a(1 - e^2)$ into Equation ?? and simplifying (try it; everything but one factor cancels) gives the energy in terms of the size of the orbit alone:

$$\varepsilon = -\frac{\mu}{2a}. \quad (4.10)$$

The energy of an orbit depends only on its semi-major axis, not on its eccentricity. A circle of radius a and a needle-thin ellipse of semi-major axis a have the same energy.

4.6 Vis-viva, circular speed, and escape speed

Put Equation ?? into the definition of ε and solve for the speed:

$$v^2 = \mu \left(\frac{2}{r} - \frac{1}{a} \right). \quad (4.11)$$

This is the *vis-viva* equation (Latin for “living force,” an old name for kinetic energy). It gives the speed at any distance r along an orbit of semi-major axis a , and it is the formula you want whenever you know where a body is and need to know how fast it should be going.

Two special cases are the ones the documentation quotes. On a circular orbit, $r = a$ always, so

$$v_{\text{circ}} = \sqrt{\frac{\mu}{r}}. \quad (4.12)$$

On a parabolic orbit, $a \rightarrow \infty$ and the $1/a$ term vanishes, so the speed that just barely escapes from distance r is

$$v_{\text{esc}} = \sqrt{\frac{2\mu}{r}} = \sqrt{2} v_{\text{circ}}. \quad (4.13)$$

Now the rules of thumb in the two-body vignette can be made exact. Suppose you place a body at distance r from the central mass and launch it perpendicular to the radius at speed $v = k v_{\text{circ}}$. Then $h = rv$, so $p = h^2/\mu = r^2 v^2/\mu = k^2 r$. The launch point has $\dot{r} = 0$, so it is either periapsis or apoapsis, and Equation ?? gives $r = p/(1 \pm e)$, or

$$e = |k^2 - 1|. \quad (4.14)$$

If $k > 1$ the launch point is periapsis and the orbit swings outward; if $k < 1$ it is apoapsis and the orbit dips inward; $k = 1$ is a circle; $k = \sqrt{2}$ gives $e = 1$, a parabola. A planet launched at 1.2 times circular speed has $e = 0.44$; at 1.4 times, $e = 0.96$, a sliver away from escape. We will check both numbers in a moment.

4.7 Kepler's third law

The area of an ellipse is πab . By Equation ?? the orbit sweeps area at the constant rate $h/2$, so one full orbit takes $T = 2\pi ab/h$. Substitute $b = a\sqrt{1-e^2}$ and $h = \sqrt{\mu a(1-e^2)}$:

$$T = \frac{2\pi a^2 \sqrt{1-e^2}}{\sqrt{\mu a(1-e^2)}} = 2\pi \sqrt{\frac{a^3}{\mu}}. \quad (4.15)$$

The eccentricity has cancelled: **Kepler's third law**. The period depends on the semi-major axis and the total mass and on nothing else. Kepler's own statement, that T^2/a^3 is the same number for every planet, holds only because every planet's mass is negligible next to the Sun's, so $\mu \approx GM_\odot$ for all of them. The correction for Jupiter is one part in a thousand in μ , half that in T .

This is also the law that weighs things. Measure a moon's period and orbital radius and Equation ?? gives you the mass of its planet; this is how the masses in Appendix ?? were obtained, and it is how we know the mass of the Sun, of Jupiter, and of the black hole at the center of the Milky Way.

4.8 Checking everything against the simulation

Time to see whether the package agrees with the algebra. The derivations above are exact for two bodies; the simulation is approximate; if they match, both the physics and the numerics are in good shape. To keep the comparison honest I will compute every prediction with the full $\mu = G(M + m)$.

4.8.1 The speed sweep

Set up the vignette's star and planet, and launch the planet at several multiples of circular speed: